



Diseño e Implementación de una Base de Datos para la Gestión Integral de un Restaurante

Proyecto Final - Bases de Datos, Semestre 2026-2

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Desarrolladores:

Alejandro Arias Jiménez

Murat Calderon Almanza

Isaac Jareth Campos Cortés

Dan Benjamín Cruz Trejo

Christopher Giovanni De la O Vicente

Francisco Javier Reyes Abarca

Mayo 30, 2026



Contenido

Índice

1. Introducción	3
2. Plan de Trabajo	4
3. Diseño	5
3.1. Modelo Entidad-Relación	5
3.2. Modelo Relacional	7
4. Implementación	9
4.1. Script para crear usuarios en la Base de Datos	9
4.2. Script DDL de la Base de Datos	10
4.3. Script DQL de la Base de Datos	16
4.4. Script PGSQL	17
5. Migración de datos	22
6. Conclusiones	31

1. Introducción

El presente documento describe el análisis, diseño e implementación de una base de datos para la digitalización de las operaciones de un restaurante, como parte del proyecto final de la asignatura Bases de Datos. El restaurante requiere un sistema informático que permita gestionar eficientemente su personal, productos, ordenes de consumo y facturación, con el fin de optimizar procesos internos y mejor toma de decisiones.

El problema se ha dividido en dos partes. En la primera parte, se debe diseñar una base de datos relacional que almacene información detallada de los empleados (cocineros, administrativos y meseros), sus dependientes, los platillos y bebidas ofrecidos, las órdenes generadas, y los datos de los clientes que solicitan factura. Adicionalmente, el diseño debe cumplir con reglas de negocio como la actualización automática de datos específicos, validación de disponibilidad, generación de vistas relevantes, consultas específicas y la creación de al menos un índice justificado. También se debe aplicar el requerimiento a la generación del folio de orden y el que los atributos compuestos se deben desglosar adecuadamente. La segunda parte del problema requiere orquestar la inserción masiva de datos desde archivos de texto hacia otra base de datos, con validación de errores.

El objetivo principal es desarrollar una solución completa de bases de datos en PostgreSQL (PgSQL) que satisfaga todos los requerimientos funcionales y de diseño establecidos. Como objetivos específicos se contemplan:

- Elaborar el modelo entidad-relación y el modelo relacional que reflejan fielmente la estructura de información del restaurante.
- Implementar el esquema físico mediante un script DDL que incluya tablas, restricciones de integridad, dominios y tipos compuestos si es necesario.
- Crear los elementos de programación en la base de datos que automaticen la lógica de negocio, como la actualización de totales, validación de disponibilidad, generación de folios y vistas automáticas.
- Construir las consultas, vistas e índices solicitados, justificando su elección.
- Desarrollar un mecanismo controlado para la carga de datos desde archivos de texto hacia una base de datos secundaria, manejando posibles errores de flujo.

La solución propuesta consta de dos partes principales. En la primera, se diseñó una base de datos relacional a partir de un modelo entidad-relación y su correspondiente modelo relacional, siguiendo las reglas de negocio, normalizando las estructuras y respetando los dominios y la composición de atributos compuestos. La implementación física se realizó en PostgreSQL mediante varios scripts que dividen la construcción en secciones como DDL, DQL, DML, entre otros. Para cumplir con los requerimientos funcionales, se desarrollaron triggers y funciones que validan las condiciones solicitadas. También se definió un índice, y en materia de seguridad se planificaron distintos usuarios con diferentes privilegios y límites de conexión.

Para la segunda parte, se implementó un proceso orquestado mediante Apache Airflow que transfiere los datos de la base operativa a la base de datos final usando archivos CSV como intermediarios. El flujo consiste en cuatro etapas secuenciales: verificación de requisitos, exportación de cada tabla a CSV, validación de los archivos generados y carga en la base de destino. La carga se realiza dentro de una transacción que primero limpia la base destino, inserta los datos y ajusta las secuencias para mantener la integridad de los identificadores. Cualquier fallo en alguna etapa detiene el flujo completo y reporta la causa del error.



2. Plan de Trabajo

Para garantizar la efectiva realización de las actividades necesarias para concretar el proyecto de la mejor manera posible, como equipo decidimos dividir el trabajo en cuatro distintas fases, cada una de estas fases se relaciona con una parte del proceso de desarrollo de la base de datos. A su vez, estas fases continen distintas tareas necesarias para que dicha fase se pueda considerar terminada.

Para garantizar eficiencia y una distribución equitativa de dichas tareas, se decidió la clasificación del equipo en dos roles: Responsables y Ayudantes. Los Responsables tienen como tarea asegurarse que la tarea cumpla con lo solicitado en tiempo y forma, organizando y revisando el progreso de los Ayudantes. Cabe resaltar que, aunque las personas denominadas para una cierta tarea cargan con la mayor parte de la misma, no significa que el resto del equipo se deslindara de la misma, para todas las tareas se realiza un consenso sobre la manera de resolver dicha tarea y se comenta abiertamente sobre posibles cambios y mejoras al entregable.

Tabla de responsabilidades

A continuación se presenta una tabla en la cual se observa gráficamente la organización previamente definida. Se puede observar cada fase con sus respectivas tareas a cumplir al igual que los integrantes responsables del trabajo principal en cada tarea.

Se utilizan los nombres de cada integrante para su identificación en la primer fila.

R = Responsable **A** = Ayudante

Actividad	Alejandro	Murat	Isaac	Dan	Christopher	Javier
Fase 1 — Análisis y diseño						
Análisis de requerimientos	A	A	A	A	R	A
Diseño conceptual (MER)	A	A	R	A	A	A
Diseño lógico (MR)	A	A	A	R	A	A
Fase 2 — Implementación de la base de datos						
Diseño físico — DDL	A	R	A	A	A	A
Triggers	R	A	A	A	A	R
Stored procedures y funciones	A	R	A	A	A	R
Vistas e índices	A	A	A	A	R	A
Script de inserción de datos	R	A	A	A	A	A
Fase 3 — Migración de datos						
Orquestación de archivos	A	A	A	A	A	R
Validación del flujo	A	A	A	A	A	R
Fase 4 — Integración, documentación y exposición						
Pruebas e integración	A	A	A	A	A	R
Redacción del documento	A	A	A	A	R	A
Preparación de la exposición	R	A	A	A	A	A

Tabla 1: Matriz de responsabilidades del equipo por fases

3. Diseño

3.1. Modelo Entidad-Relación

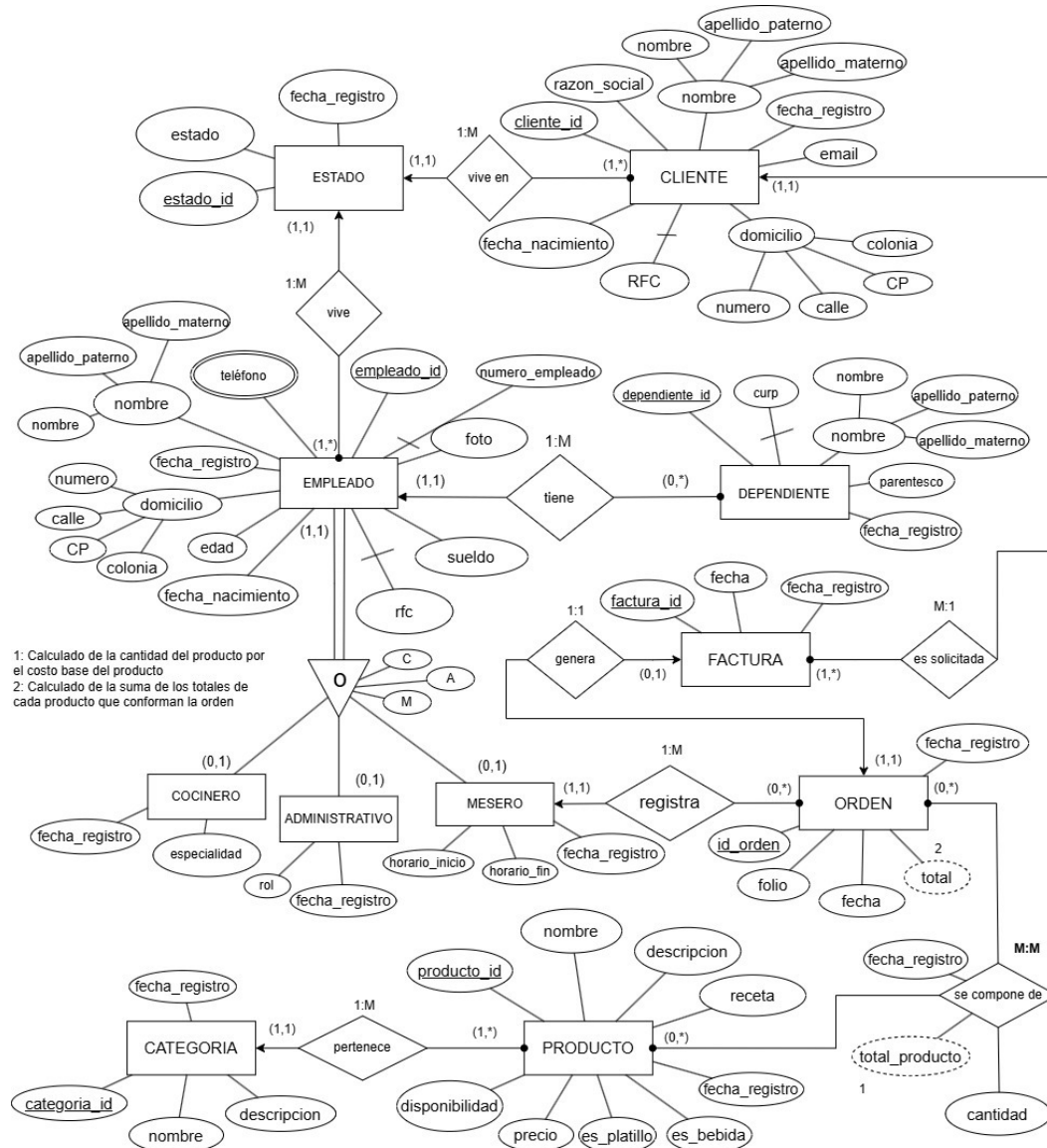


Figura 1: Modelo Entidad-Relación

Para el modelo Entidad-Relación, se revisaron los aspectos más importantes a partir de las necesidades operativas del restaurante: el personal y sus distintos puestos, los dependientes de los empleados, el catálogo de platillos y bebidas con sus categorías, el registro de órdenes con sus detalles, y la opción de facturación para los clientes.

La entidad EMPLEADO representa al personal que labora en el restaurante y almacena la información necesaria para su identificación, contacto y administración. Entre sus atributos más relevantes se encuentran empleado_id, una clave artificial utilizada como identificador único, los atributos compuestos nombre y domicilio, y el atributo multivaluado teléfono.

Adicionalmente, se estableció la relación entre EMPLEADO y DEPENDIENTE para almacenar la información de



las personas que dependen económicamente del empleado. Un empleado puede estar asociado con varios dependientes, mientras que cada dependiente pertenece únicamente a un empleado. Aunque conceptualmente podría modelarse como una entidad débil, por cuestiones de diseño y posterior transformación al modelo relacional se optó por manejarla como una entidad fuerte.

Para representar las distintas funciones desempeñadas dentro de la organización, la entidad EMPLEADO se especializa en los subtipos MESERO, COCINERO y ADMINISTRATIVO. Esta especialización se definió como total u obligatoria, lo que implica que todo empleado debe pertenecer al menos a uno de los subtipos. Asimismo, se definió como traslape, permitiendo que un mismo empleado pueda desempeñar más de una función dentro del negocio. La combinación de estas restricciones garantiza que todos los empleados tengan al menos un rol asignado, manteniendo la flexibilidad necesaria para representar las reglas de negocio establecidas.

Para el catálogo del restaurante, se definió la entidad PRODUCTO, la cual unifica tanto platillos como bebidas. Esto evita duplicar atributos comunes como nombre, descripción, precio, disponibilidad y receta. Para distinguir el tipo de producto, se incluyeron los atributos `es_platillo` y `es_bebida` que indican si se trata de un platillo o una bebida. Cada producto pertenece a una sola CATEGORÍA, relación que se modeló como uno a muchos, ya que una categoría puede agrupar varios productos, pero cada producto solo tiene una categoría. La categoría almacena su nombre y una descripción.

La entidad ORDEN representa cada transacción de venta. Se definió una clave artificial `orden_id` como identificador único. Adicionalmente, almacena la fecha y hora de la operación, el total a pagar, y establece una relación de uno a muchos con la entidad MESERO (subtipo de EMPLEADO), ya que un mesero puede levantar muchas órdenes, pero cada orden es levantada por un solo mesero.

Entre ORDEN y PRODUCTO existe una relación de muchos a muchos, ya que una orden puede contener varios productos y un producto puede aparecer en muchas órdenes. Esta relación posee atributos propios: la cantidad de cada producto solicitado y el `total_producto`, el cual se modeló como un atributo derivado, pues puede calcularse como la cantidad multiplicada por el precio del producto (almacenado en la entidad PRODUCTO) en el momento de la venta. De esta forma, cada vez que se agrega o modifica un producto en una orden, se puede recalcular automáticamente el total de la orden, cumpliendo con el requerimiento.

Para atender la facturación, se definió la entidad CLIENTE, que contiene la información fiscal del cliente: nombre, RFC, razón social, domicilio compuesto, email y fecha de nacimiento. No todos los clientes solicitan factura, por lo que la existencia de un cliente implica necesariamente una factura. Cuando un cliente requiere facturar una orden, se genera la entidad FACTURA, la cual establece una relación de muchos a uno con CLIENTE (un cliente puede tener varias facturas y una factura pertenece a un solo cliente) y una relación de uno a uno con ORDEN (una orden puede tener una sola factura y una factura pertenece a una sola orden). Así se satisface el requerimiento de almacenar los datos fiscales solo cuando sea necesario.

La entidad ESTADO se generó para tener un catálogo de estados pues los comparten tanto CLIENTE como EMPLEADO. Además, todas las entidades incluyeron un atributo `fecha_registro`, el cual se explica más adelante su uso.

3.2. Modelo Relacional

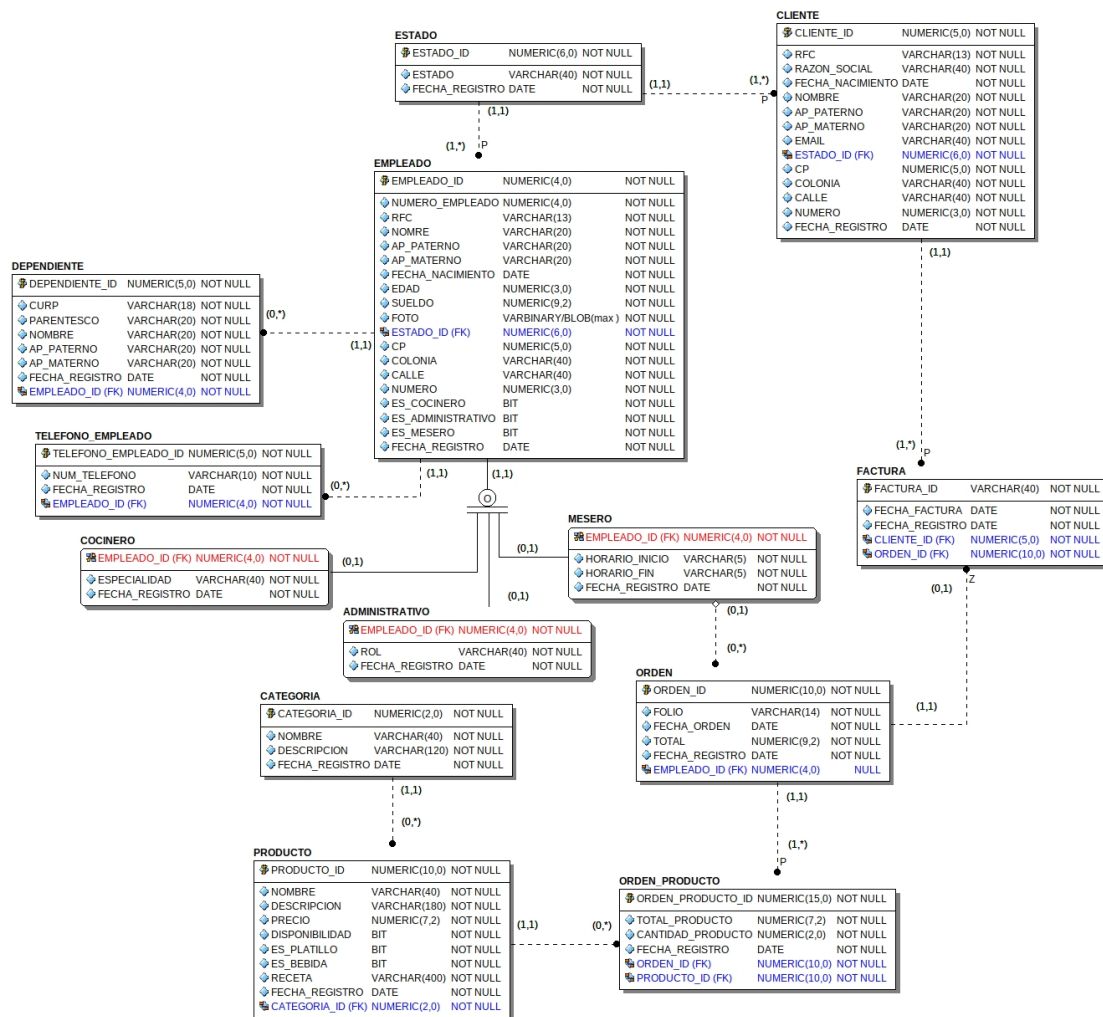


Figura 2: Modelo Relacional

Con base en el diseño presentado en el Modelo Entidad-Relación, se definió el Modelo Relacional que representa la base de datos. Para su diseño se tomaron en cuenta las restricciones de negocio del restaurante, asegurando que fueran implementadas adecuadamente en el MR.

Se consideraron la cardinalidad de las relaciones y los tipos de dato de cada atributo para representar correctamente la información requerida. De igual manera, se hizo énfasis en el uso de claves artificiales en cada tabla, por sus beneficios en eficiencia e integridad de los datos.

Por último, se destaca la presencia del atributo fecha_registro en cada tabla, el cual registra la última fecha en que se realizó una operación DML sobre el registro. Este atributo se planteó como una solución inicial al filtrado para la migración diaria de datos hacia la base de datos final (finalmente no se utilizó pero se conserva en las tablas para mantener la propuesta de implementación original).

La tabla EMPLEADO se identifica mediante empleado_id como clave artificial. Almacena los datos de identificación del personal: numero_empleado, rfc, el nombre compuesto (nombre, ap_paterno, ap_materno), fecha_nacimiento, edad, sueldo y foto. El domicilio se desglosa en cp, colonia, calle y numero, complementado por la llave foránea



estado_id.

Para representar los tipos de puesto, se incluyen tres banderas booleanas: `es_cocinero`, `es_administrativo` y `es_mesero`, lo que permite que un empleado pueda ocupar varios puestos simultáneamente. La relación entre supertipo y subtipos conecta EMPLEADO con COCINERO (con atributo especialidad), ADMINISTRATIVO (con atributo rol) y MESERO (con atributos `horario_inicio` y `horario_fin`). Gráficamente se representa con el círculo de herencia, donde la "O" indica traslape y las líneas dobles la obligatoriedad. La cardinalidad (1,1) en el supertipo y (0,1) en cada subtipo refleja que todo empleado debe pertenecer a al menos un tipo, sin estar restringido a solo uno.

Para el registro de los teléfonos, al ser un atributo multivaluado, se generó la tabla TELEFONO_EMPLEADO, identificada con la clave artificial `telefono_empleado_id`. La llave foránea `empleado_id` reside en esta tabla para permitir que un empleado tenga múltiples números registrados. La cardinalidad es (0,M) en TELEFONO_EMPLEADO y (1,1) en EMPLEADO.

La relación con DEPENDIENTE se representa mediante la llave foránea `empleado_id` en dicha tabla, con cardinalidad (0,M) en DEPENDIENTE y (1,1) en empleado. Los atributos registrados son `curp`, `parentesco`, `nombre`, `ap_paterno` y `ap_materno`.

La tabla ESTADO funciona como catálogo de los estados de la república, referenciado mediante llave foránea tanto en EMPLEADO como en CLIENTE. La cardinalidad es (1,1) hacia ESTADO y (1,M) desde este, dado que varias personas pueden residir en el mismo estado.

La tabla CLIENTE se relaciona con ESTADO y con FACTURA; su clave funge como llave foránea en esta última. La cardinalidad es (1,1) en CLIENTE y (1,M) en FACTURA, pues un cliente puede solicitar varias facturas, pero cada factura pertenece a un único cliente.

Además, FACTURA contiene la clave de ORDEN como llave foránea, con cardinalidad (0,1) en FACTURA y (1,1) en ORDEN, lo que refleja que una orden puede no tener factura o tener a lo sumo una, y que cada factura corresponde exactamente a una orden.

La tabla ORDEN se relaciona con MESERO y con la tabla intermedia ORDEN_PRODUCTO. Con MESERO la cardinalidad es (0,M) en ORDEN y (0,1) en MESERO, ya que un mesero puede atender múltiples órdenes y una orden puede quedar sin mesero asignado. Con ORDEN_PRODUCTO la cardinalidad es (1,1) en ORDEN y (1,M) en ORDEN_PRODUCTO; la llave foránea `orden_id` reside en la tabla intermedia, necesaria para resolver la relación muchos a muchos entre órdenes y productos.

En ORDEN_PRODUCTO se almacenan los atributos `total_producto` y `cantidad_producto`, específicos de la relación, junto con las llaves foráneas de ORDEN y PRODUCTO.

La tabla PRODUCTO se relaciona con ORDEN_PRODUCTO con cardinalidad (0,M), ya que pueden existir productos no incluidos en ninguna orden. Con CATEGORIA la cardinalidad es (1,1) en PRODUCTO y (0,M) en CATEGORIA, pues un producto debe pertenecer a exactamente una categoría, mientras que una categoría puede no tener productos asignados. Destacan las banderas booleanas `es_platillo` y `es_bebida` para distinguir el tipo de producto, y el atributo `disponibilidad` para indicar si se encuentra en existencia.

4. Implementación

4.1. Script para crear usuarios en la Base de Datos

```
1 create user admin_res with
2   password 'admin_res'
3   connection limit 2;
4
5 grant all privileges on database Restaurante_Dia to admin_res;
6 grant all privileges on schema public to admin_res;
7 grant all privileges on all tables in schema public to admin_res;
8 grant all privileges on all sequences in schema public to admin_res;
9 grant all privileges on all functions in schema public to admin_res;
```

1: Creación de usuario admin y asignación de permisos

Usuario admin_res: Es el usuario administrador del restaurante. Cuenta con todos los privilegios sobre la base de datos. Su límite de conexión es de 2 sesiones simultáneas, lo que lo restringe a uso administrativo exclusivo. Este usuario se crea antes de ejecutar el script de DDL. De esta forma, el script DDL puede ser ejecutado por el usuario admin_res, garantizando que sea el propietario de todos los objetos de la base de datos.

```
1 create user administrativo with
2   password 'administrativo'
3   connection limit 3;
4
5 create user mesero with
6   password 'mesero'
7   connection limit 4;
8
9 grant connect on database Restaurante_Dia to administrativo;
10 grant usage, create on schema public to administrativo;
11 grant select, insert, update, delete on all tables in schema public to administrativo;
12 grant usage on all sequences in schema public to administrativo;
13 grant execute on all functions in schema public to administrativo;
14 grant execute on all procedures in schema public to administrativo;
15
16 grant connect on database Restaurante_Dia to mesero;
17 grant usage on schema public to mesero;
18 grant usage on all sequences in schema public to mesero;
19 grant select, insert, update on orden in schema public to mesero;
20 grant select, insert, update on orden_producto in schema public to mesero;
21 grant select, insert, update on factura in schema public to mesero;
22 grant select, insert, update on cliente in schema public to mesero;
23 grant select on estado in schema public to mesero;
```

2: Creación de usuarios administrativo y mesero y asignación de permisos

Usuario administrativo: Rol destinado al personal administrativo del restaurante. Tiene amplios permisos de consulta y modificación sobre toda la base de datos, pero no puede crear objetos a nivel de base de datos. Su límite de conexión es de 3 sesiones simultáneas.

Usuario mesero: Rol destinado a los meseros del restaurante. Sus permisos están estrictamente limitados a las operaciones necesarias para realizar su trabajo: registrar órdenes, agregar productos a una orden, registrar clientes para facturación y consultar el catálogo de productos disponibles. Su límite de conexión es de 4 sesiones simultáneas.

4.2. Script DDL de la Base de Datos

A continuación se presenta el script DDL implementado para la creación de la base de datos del restaurante. Este script incluye la definición de tipos compuestos, la creación de tablas con sus respectivas columnas, restricciones de integridad, así como la generación de la secuencia utilizada para el folio de las órdenes con el formato solicitado. Todo el diseño ha sido normalizado hasta la tercera forma normal para garantizar la consistencia y evitar redundancias.

El campo fecha_registro es una columna de auditoria común a todas las tablas. Su propósito es registrar automáticamente la fecha en que cada registro fue manipulado en la base de datos. Se define con el tipo de date y el valor por defecto es now. Este campo es útil para trazabilidad, auditoria y para poder filtrar información por día de modificación sin depender de otras columnas.

```
1 create table estado(
2     estado_id    numeric(6,0),
3     estado       varchar(40)    not null,
4     fecha_registro date          not null default now(),
5     constraint estado_pk primary key (estado_id)
6 );
```

3: Tabla estado

Tabla catálogo que almacena los estados de la república. Se creó a partir del modelo relacional, donde el atributo domicilio fue desglosado en sus componentes, incluyendo estado_id como llave foránea en las tablas que requieren domicilio.

```
1 create table empleado (
2     empleado_id    numeric(4,0),
3     numero_empleado numeric(4,0)    not null,
4     rfc             varchar(13)     not null,
5     nombre          varchar(20)     not null,
6     ap_paterno      varchar(20)     not null,
7     ap_materno      varchar(20)     not null,
8     fecha_nacimiento date           not null,
9     edad            numeric(2,0)    not null,
10    sueldo           numeric(9,2)    not null,
11    foto             bytea           not null,
12    estado_id        numeric(6,0)    not null,
13    cp               numeric(6,0)    not null,
14    colonia          varchar(40)     not null,
15    calle            varchar(40)     not null,
16    numero           numeric(3,0)    not null,
17    es_cocinero       boolean         not null,
18    es_administrativo boolean         not null,
19    es_mesero         boolean         not null,
20    fecha_registro   date             not null default now(),
21    constraint empleado_pk primary key(empleado_id),
22    constraint empleado_rfc_uk unique(rfc),
23    constraint numero_empleado_uk unique(numero_empleado),
24    constraint empleado_tipo_chk check (
25        es_cocinero = true or es_administrativo = true or es_mesero = true
26    ),
27    constraint empleado_estado foreign key(estado_id) references estado(estado_id)
28 );
```

4: Tabla empleado

Tabla principal del personal del restaurante. Los atributos de domicilio fueron desglosados directamente en esta tabla. Se incluyen banderas booleanas para indicar que tipo(s) de puesto ocupa el empleado, validando mediante

una restricción check que asegura que al menos pertenezca a un subtipo.

```
1 create table telefono_empleado (
2     telefono_empleado_id    numeric(5,0),
3     num_telefono            varchar(10)    not null,
4     empleado_id             numeric(4,0)   not null,
5     fecha_registro          date           not null default now(),
6     constraint telefono_empleado_pk primary key(telefono_empleado_id),
7     constraint telefono_empleado_fk foreign key(empleado_id) references empleado(
8         empleado_id) on delete cascade
9 );
```

5: Tabla telefono_empleado

Tabla que permite almacenar múltiples números telefónicos por empleado. Se aplicó la opción on delete cascade para que al eliminar un empleado, se eliminen automáticamente sus teléfonos asociados.

```
1 create table dependiente (
2     dependiente_id    numeric(5,0),
3     curp               varchar(18)    not null,
4     parentesco         varchar(20)    not null,
5     nombre             varchar(20)    not null,
6     ap_paterno         varchar(20)    not null,
7     ap_materno         varchar(20)    not null,
8     empleado_id        numeric(4,0)   not null,
9     fecha_registro     date            not null default now(),
10    constraint dependiente_pk primary key(dependiente_id),
11    constraint dependiente_empleado_fk foreign key(empleado_id)
12        references empleado(empleado_id) on delete cascade,
13    constraint dependiente_curp_uk unique(curp)
14 );
```

6: Tabla dependiente

Almacena a las personas dependientes de cada empleado. La CURP se definió como única para evitar duplicados, al igual que los teléfonos, se usa on delete cascade para mantener la integridad referencial.

```
1 create table cocinero (
2     empleado_id        numeric(4,0),
3     especialidad        varchar(40)    not null,
4     fecha_registro      date            not null default now(),
5     constraint cocinero_empleado_fk foreign key (empleado_id)
6         references empleado(empleado_id) on delete cascade on update cascade,
7     constraint cocinero_empleado_pk primary key (empleado_id)
8 );
```

7: Tabla cocinero

Tabla que especializa a los empleados como cocineros. La llave primaria es a la vez la llave foránea hacia el empleado, modelando una relación de herencia. Se usó on delete cascade y on update cascade para reflejar cambios desde la tabla padre.

```
1 create table administrativo (
2     empleado_id        numeric(4,0),
3     rol                 varchar(40)    not null,
4     fecha_registro      date            not null default now(),
5     constraint administrativo_empleado_fk foreign key (empleado_id)
6         references empleado(empleado_id) on delete cascade on update cascade,
7     constraint administrativo_empleado_pk primary key (empleado_id)
8 );
```

```
8 );
```

8: Tabla administrativo

Similar a cocinero, especializa a los empleados administrativos, añadiendo el atributo rol. Se aplicó el mismo patrón de herencia con llave primaria y foránea hacia empleado.

```
1 create table mesero (
2     empleado_id      numeric(4,0),
3     horario_inicio   varchar(5) not null,
4     horario_final    varchar(5) not null,
5     fecha_registro   date      not null default now(),
6     constraint mesero_empleado_fk foreign key (empleado_id)
7         references empleado(empleado_id) on delete cascade on update cascade,
8     constraint mesero_empleado_pk primary key (empleado_id)
9 );
```

9: Tabla mesero

Especialización para meseros, incluyendo su horario de trabajo. Se optó por tipo varchar(5) para almacenar horas en formato HH:MM. Se aplicó el mismo patrón de herencia con llave primaria y foránea hacia empleado.

```
1 create table categoria(
2     categoria_id      numeric(2,0)      not null,
3     nombre            varchar(40)       not null,
4     descripcion       varchar(120)      not null,
5     fecha_registro    date              not null default now(),
6     constraint categoria_pk primary key(categoria_id)
7 );
```

10: Tabla categoria

Catálogo que clasifica los productos del restaurante. Es una tabla simple que se implementa directamente desde el modelo relacional sin modificaciones significativas.

```
1 create table orden(
2     orden_id          numeric(10,0)     not null,
3     folio              varchar(14)       not null,
4     fecha_orden        timestampz        not null,
5     total              numeric(9,2)      not null,
6     empleado_id        numeric(4,0)      null,
7     fecha_registro     date              not null default now(),
8     constraint orden_pk primary key(orden_id),
9     constraint orden_empleado_id_fk foreign key(empleado_id)
10         references mesero(empleado_id) on delete set null on update cascade
11 );
```

11: Tabla orden

Registra las órdenes de consumo realizadas en el restaurante. La llave foránea empleado_id apunta a la tabla mesero, ya que solo un mesero puede levantar una orden. Se usó on delete set null para conservar el historial de órdenes incluso si el mesero es eliminado.

```
1 create table producto(
2     producto_id       numeric(10,0)     not null,
3     nombre            varchar(40)       not null,
4     descripcion       varchar(180)      not null,
5     precio            numeric(7,2)      not null,
6     disponibilidad    boolean           not null,
```

```

7  es_platillo      boolean          not null,
8  es_bebida       boolean          not null,
9  receta          varchar(400)     not null,
10 categoria_id    numeric(2,0)     not null,
11 fecha_registro  date              not null default now(),
12 constraint producto_pk primary key(producto_id),
13 constraint producto_categoria_id_fk foreign key(categoria_id)
14 references categoria(categoria_id) on update cascade,
15 constraint producto_es_tipo_chk
16 check((es_platillo = true and es_bebida = false)
17        or (es_platillo = false and es_bebida = true))
18 );

```

12: Tabla producto

Almacena platillos y bebidas ofrecidas por el restaurante. Se utilizaron dos banderas booleanas con una restricción check que asegura que un producto sea exactamente uno de los dos tipos, nunca ambos o ninguno.

```

1  create table orden_producto(
2  orden_producto_id numeric(15,0)  not null,
3  total_producto   numeric(2,0)    not null,
4  cantidad_producto numeric(2,0)    not null,
5  orden_id         numeric(10,0)    not null,
6  producto_id      numeric(10,0)    not null,
7  fecha_registro   date              not null default now(),
8  constraint orden_producto_pk primary key(orden_producto_id),
9  constraint orden_producto_orden_id_fk
10 foreign key(orden_id) references orden(orden_id),
11 constraint orden_producto_producto_id_fk
12 foreign key(producto_id) references producto(producto_id)
13 );

```

13: Tabla orden_producto

Tabla puente que relaciona órdenes con productos (relación muchos a muchos). Almacena la cantidad solicitada y el total del producto. Esa tabla es esencial para calcular los totales de cada orden y para los triggers solicitados en los requerimientos.

```

1  create table cliente(
2  cliente_id      numeric(5,0),
3  rfc             varchar(13)       not null,
4  razon_social    varchar(40)       not null,
5  fecha_nacimiento date            not null,
6  nombre         varchar(20)        not null,
7  ap_paterno     varchar(20)        not null,
8  ap_materno     varchar(20)        not null,
9  email          varchar(40)        not null,
10 estado_id      numeric(6,0)       not null,
11 cp             numeric(5,0)       not null,
12 colonia        varchar(40)        not null,
13 calle          varchar(40)        not null,
14 numero         numeric(3,0)       not null,
15 fecha_registro  date              not null default now(),
16 constraint cliente_pk primary key(cliente_id),
17 constraint cliente_estado_fk foreign key(estado_id)
18 references estado(estado_id),
19 constraint cliente_rfc_uk unique(rfc)
20 );

```

14: Tabla cliente

Almacena la información de los clientes que pueden solicitar factura, incluye desglose completo del domicilio y el nombre compuesto, además del RFC con restricción de unicidad.

```
1 create table factura (  
2     factura_id      numeric(7,0),  
3     fecha_factura   timestampz not null,  
4     cliente_id      numeric(4,0) not null,  
5     orden_id       numeric(10,0) not null,  
6     fecha_registro  date         not null default now(),  
7     constraint factura_pk primary key (factura_id),  
8     constraint factura_cliente_fk foreign key (cliente_id)  
9         references cliente(cliente_id),  
10    constraint factura_orden_fk foreign key (orden_id)  
11        references orden(orden_id)  
12 );
```

15: Tabla factura

Relaciona una orden con un cliente que solicita factura. Una orden sólo puede tener una factura asociada debido a su relación uno a uno.

```
1 create sequence estado_id_seq  
2 start with 1  
3 increment by 1  
4 minvalue 1  
5 maxvalue 999999;
```

16: Secuencia estado_id_seq

Secuencia utilizada para generar automáticamente los valores de la llave primaria estado.id en la tabla estado. Se configuró con inicio en 1 y un máximo de 999,999, suficiente para los estados del país.

```
1 create sequence empleado_id_seq  
2 start with 1  
3 increment by 1  
4 minvalue 1  
5 maxvalue 9999;
```

17: Secuencia empleado_id_seq

Genera los identificadores únicos para la tabla empleado. El límite máximo de 9,999 empleados se considera adecuado para las operaciones del restaurante.

```
1 create sequence telefono_empleado_id_seq  
2 start with 1  
3 increment by 1  
4 minvalue 1  
5 maxvalue 99999;
```

18: Secuencia telefono_empleado_id_seq

Secuencia para la llave primaria de la tabla telefono_empleado.

```
1 create sequence dependiente_id_seq  
2 start with 1  
3 increment by 1  
4 minvalue 1  
5 maxvalue 99999;
```

19: Secuencia dependiente_id_seq

Genera los identificadores para los dependientes de los empleados en la tabla 'Dependiente'.

```
1 create sequence categoria_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 99;
```

20: Secuencia categoria_id_seq

Secuencia para la tabla catálogo categoria. Con un máximo de 99, es suficiente para clasificar todos los productos del restaurante.

```
1 create sequence factura_folio_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 99999;
```

21: Secuencia factura_folio_seq

Esta secuencia se utiliza exclusivamente para generar la parte numérica del folio de las órdenes, combinándose con el prefijo ORD- para el formato solicitado.

```
1 create sequence orden_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 9999999999;
```

22: Secuencia orden_id_seq

Genera los identificadores numéricos para la tabla orden.

```
1 create sequence producto_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 9999999999;
```

23: Secuencia producto_id_seq

Secuencia para la llave primaria de la tabla producto.

```
1 create sequence orden_producto_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 9999999999;
```

24: Secuencia orden_producto_id_seq

Genera los identificadores para cada registro puente orden_producto. El alto límite responde a que cada orden puede tener múltiples productos, por lo que el número de registros puede ser considerable.

```
1 create sequence cliente_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
```

```
5 maxvalue 99999;
```

25: Secuencia cliente_id_seq

Secuencia para la tabla cliente, con capacidad para 99,999 clientes registrados que pueden solicitar factura.

```
1 create sequence factura_id_seq
2 start with 1
3 increment by 1
4 minvalue 1
5 maxvalue 9999999;
```

26: Secuencia factura_id_seq

Genera los identificadores únicos para la tabla factura, con un límite de 9,999,999 facturas emitidas.

```
1 create index ind_orden_fecha_orden on orden(fecha_orden);
```

27: Creación del índice sobre fecha_orden en la tabla orden

Se creó un índice sobre la columna fecha_orden de la tabla orden. Este índice acelera las consultas que filtran por rangos de fechas o fechas específicas, mejorando el rendimiento a medida que crece el volumen de órdenes almacenadas.

4.3. Script DQL de la Base de Datos

```
1 create or replace view platillo_mas_vendido as
2   with cantidades as (
3     select p.producto_id,
4           sum(op.cantidad_producto) as cantidad_vendida_producto
5     from orden_producto op
6     join producto p
7       on p.producto_id = op.producto_id
8     where p.es_platillo = true
9     group by producto_id)
10  select p.producto_id, p.nombre, p.descripcion, p.precio, p.disponibilidad
11  from producto p
12  join cantidades c
13    on p.producto_id = c.producto_id
14  where p.cantidad_vendida_producto = (select max(cantidad_vendida_producto)
15                                     from c);
```

28: Vista platillo_mas_vendido

Esta vista muestra todos los detalles del platillo más vendido en el restaurante. La implementación utiliza una subconsulta en el JOIN para calcular la cantidad total vendida de cada producto. Posteriormente, otra subconsulta obtiene la cantidad máxima vendida entre todos los platillos. Finalmente, se filtran aquellos productos cuya cantidad vendida coincida con dicho máximo.

```
1 create or replace view productos_no_disponibles as
2   select nombre
3   from producto
4   where disponibilidad = false;
```

29: Vista productos_no_disponibles

Vista que permite obtener el nombre de todos aquellos productos (tanto platillos como bebidas) que actualmente no se encuentran disponibles para la venta.

4.4. Script PGSQL

```

1 create or replace function fn_validar_y_calcular_total_producto()
2 returns trigger as $$
3 declare
4     v_disponibilidad_producto producto.disponibilidad%type;
5     v_precio_producto producto.precio%type;
6 begin
7     select p.disponibilidad, p.precio
8     into v_disponibilidad_producto, v_precio_producto
9     from producto p
10    where p.producto_id = new.producto_id;
11
12    if v_disponibilidad_producto is false then
13        raise exception 'No hay disponibilidad de producto, por lo que no se puede
14        agregar.'
15        using errcode = 'E0001';
16    end if;
17
18    new.total_producto := new.cantidad_producto * v_precio_producto;
19
20    return new;
21 end;
22 $$ language plpgsql;
23
24 create trigger trg_validar_y_calcular_total_producto
25 before insert or update of producto_id, cantidad_producto
26 on orden_producto
27 for each row
28 execute function fn_validar_y_calcular_total_producto();

```

30: Función para validar disponibilidad y calcular total por producto

Esta función se ejecuta como trigger de tipo BEFORE cada vez que se intenta insertar o actualizar un registro en la tabla orden_producto, pero únicamente cuando cambien las columnas producto_id o cantidad_producto. Al ser BEFORE, la función tiene la oportunidad de modificar los valores del nuevo registro (NEW) antes de que se escriban en la tabla. Primero consulta la tabla producto para obtener la disponibilidad y el precio del producto que se está agregando. Si la disponibilidad es false, se lanza la excepción con un código de error personalizado E0001, lo que impide que la operación se complete. En caso de que el producto esté disponible, la función calcula el total correspondiente a esa línea multiplicando la cantidad solicitada por el precio unitario y asigna el resultado a NEW.total_producto. Finalmente retorna NEW, lo que permite que la inserción o actualización continúe con el total ya calculado. De esta forma se garantiza que ningún producto no disponible pueda ser añadido a una orden y que el subtotal por producto siempre sea correcto.

```

1 create or replace function fn_actualizar_total_orden()
2 returns trigger as $$
3 begin
4     update orden
5     set total = (
6         select sum(op.total_producto)
7         from orden_producto op
8         where op.orden_id = new.orden_id
9     )
10    where orden_id = new.orden_id;
11
12    return new;
13 end;
14 $$ language plpgsql;

```

```
15
16 create trigger trg_actualizar_total_orden
17 after insert or update of total_producto, cantidad_producto, producto_id
18 on orden_producto
19 for each row
20 execute function fn_actualizar_total_orden();
```

31: Función para actualizar el total de la orden

Esta función se ejecuta AFTER de que se inserte o actualice un registro en orden_producto. A diferencia del trigger anterior, aquí se necesita que la modificación ya haya ocurrido para poder recalcular el total generado en la orden. El trigger se dispara cuando cambia total_producto, cantidad_producto o producto_id, es decir, cualquier cambio que afecte el valor monetario de una línea de detalle. La función realiza la actualización sobre la tabla orden, específicamente sobre la fila cuyo orden_id coincide con la línea modificada. El nuevo total se calcula mediante una subconsulta que suma todos los total_producto de los detalles asociados a esa orden. Al ser un trigger AFTER, no modifica la fila actual de orden_producto sino que actualiza la cabecera de la orden. Esto asegura que el campo total de la tabla orden siempre refleje la suma correcta de sus productos, cumpliendo con el requerimiento de actualizar los totales cada vez que se agregue un producto a la orden.

```
1 create or replace function fn_reporte_ordenes_mesero_dia(
2     p_numero_empleado empleado.numero_empleado%type
3 ) returns void as $$
4 declare
5     v_empleado_id empleado.empleado_id%type;
6     v_is_mesero numeric(1,0);
7     v_numero_ordenes numeric(3,0);
8     v_total numeric(9,2);
9 begin
10     select e.empleado_id
11     into v_empleado_id
12     from empleado e
13     where e.numero_empleado = p_numero_empleado;
14
15     select count(m.empleado_id)
16     into v_is_mesero
17     from empleado e
18     left join mesero m
19         on e.empleado_id = m.empleado_id
20     where e.numero_empleado = p_numero_empleado;
21
22     if v_is_mesero = 0 then
23         raise exception 'El numero de empleado dado no es de un mesero.'
24         using errcode = 'P0002';
25     end if;
26
27     select count(*)
28     into v_numero_ordenes
29     from orden o
30     where o.empleado_id = v_empleado_id
31         and o.fecha_orden::date = current_date;
32
33     select sum(o.total)
34     into v_total
35     from orden o
36     where o.empleado_id = v_empleado_id
37         and o.fecha_orden::date = current_date;
38
39     raise notice 'Cantidad de ordenes: %, total pagado: %',
40         v_numero_ordenes, v_total;
```

```
41 end;  
42 $$ language plpgsql;
```

32: Función para reporte de órdenes por mesero en el día actual

Esta función recibe como parámetro el numero.empleado. Primero obtiene el empleado.id interno a partir de ese número. Luego realiza un left join entre empleado y mesero para determinar si existe un registro en la tabla mesero correspondiente a ese empleado, el count de mesero.empleado_id id será 1 si es mesero o 0 en caso contrario. Si el resultado es 0, la función lanza una excepción con el mensaje indicando que el número de empleado no pertenece a un mesero. En caso afirmativo, ejecutar dos consultas sobre la tabla orden: una cuenta el número de órdenes donde empleado.id sea el del mesero y cuya fecha sea igual al día actual; la otra suma el campo total de esas mismas órdenes. Finalmente, utiliza RAISE NOTICE para mostrar en la salida del servidor la cantidad de órdenes y el monto total pagado.

```
1  
2 create or replace function ventas_en_intervalo(  
3     p_fecha_ini date, p_fecha_fin date,  
4     out total_ventas numeric, out monto_total numeric)  
5 $$ as  
6 begin  
7     select count(*), sum(total) into total_ventas, monto_total  
8     from orden  
9     where fecha_orden >= p_fecha_ini and fecha_orden < p_fecha_fin + 1;  
10 end;  
11 $$ language plpgsql;  
12  
13  
14 create or replace function ventas_en_intervalo(  
15     p_fecha date, out total_ventas numeric, out monto_total numeric)  
16 $$ as  
17 begin  
18     select count(*), sum(total) into total_ventas, monto_total  
19     from orden  
20     where fecha_orden >= p_fecha and fecha_orden < (p_fecha + 1);  
21 end;  
22 $$ language plpgsql;
```

33: Funciones para ventas en intervalo de fechas

Se han creado dos variantes de la función ventas.en.intervalo utilizando el mecanismo de sobrecarga de funciones en PostgreSQL. La primera versión recibe dos parámetros tipo date y define dos parámetros de salida, por lo que no es necesario declarar explícitamente un tipo de retorno. La consulta cuenta el número de órdenes y suma el total de las mismas cuya fecha.orden sea mayor o igual a la fecha de inicio y estrictamente menor a la fecha de fin mas un día. Sumar un día a la fecha final permite incluir todas las órdenes del día completo, independientemente de la hora, sin necesidad de convertir a timestamp. La segunda versión recibe una sola fecha y realiza la misma operación pero el intervalo que va desde esa fecha hasta el día siguiente. Ambas funciones devuelven directamente el número total de órdenes y el monto acumulado.

```
1 create or replace procedure factura_orden (p_orden_id numeric)  
2 language plpgsql as $$  
3 declare  
4     v_orden_datos record;  
5     cur_factura cursor for  
6     select op.total_producto, op.cantidad_producto,  
7     p.nombre as nombre_producto, p.precio, p.es_platillo, p.es_bebida  
8     from orden o  
9     join orden_producto op on o.orden_id = op.orden_id  
10    join producto p on op.producto_id = p.producto_id
```

```

11     where o.orden_id = p_orden_id
12     order by p.es_platillo desc, p.es_bebida desc;
13 v_orden_cliente record;
14 begin
15     select o.folio, o.fecha_orden, o.total,
16            f.fecha_factura,
17            c.nombre, c.ap_paterno, c.ap_materno, c.rfc
18     into v_orden_cliente
19     from orden o
20     join factura f on o.orden_id = f.orden_id
21     join cliente c on f.cliente_id = c.cliente_id
22     where o.orden_id = p_orden_id;
23
24     if not found then
25         raise notice 'No existe la orden o no tiene factura';
26         return;
27     end if;
28
29     raise notice '=====';
30     raise notice '                Factura orden                ';
31     raise notice '=====';
32     raise notice 'Folio: %', v_orden_cliente.folio;
33     raise notice 'Fecha orden: %', v_orden_cliente.fecha_orden;
34
35     open cur_factura;
36     loop
37         fetch cur_factura into v_orden_datos;
38         exit when not found;
39         raise notice 'Producto: %, Precio unitario: %',
40            v_orden_datos.nombre_producto, v_orden_datos.precio;
41         raise notice '    Cantidad: %, Total por producto: %',
42            v_orden_datos.cantidad_producto, v_orden_datos.total_producto;
43     end loop;
44     close cur_factura;
45
46     raise notice 'Total de la orden: $%', v_orden_cliente.total;
47     raise notice 'Factura generada el: %', v_orden_cliente.fecha_factura;
48     raise notice '=====';
49     raise notice '                Datos del cliente';
50     raise notice '=====';
51     raise notice 'Nombre: % % %', v_orden_cliente.nombre,
52            v_orden_cliente.ap_paterno, v_orden_cliente.ap_materno;
53     raise notice 'RFC: %', v_orden_cliente.rfc;
54 end;
55 $$;

```

34: Procedimiento para mostrar factura de una orden

Este procedimiento recibe el identificador de una orden y genera una salida formateada que simula una factura. Primero se declara un cursor `cur_factura` que extrae los detalles de todos los productos asociados a la orden, incluyendo cantidad, precio unitario, total por producto y el nombre del producto, los resultados se ordenan para mostrar primero los platillos y luego las bebidas. También se declara un registro `v_orden_cliente` que contendrá los datos de la cabecera de la orden, la factura y el cliente. La consulta principal realiza tres JOIN: orden con factura para obtener el folio, fecha de orden, total y fecha de factura, y factura con cliente para obtener los datos del cliente. Si la orden no existe o no tiene factura asociada, la variable `FOUND` será falsa y se lanza un aviso terminando el procedimiento. En caso contrario, se imprime un encabezado con el folio y la fecha. Luego se abre el cursor y se recorre fila por fila, mostrando cada producto, su precio unitario, la cantidad y el subtotal. Al final se imprime el total de la orden, la fecha de generación de la factura y los datos completos del cliente. Todo se realiza mediante `RAISE NOTICE`.

Actualización fecha de registro

Durante la fase de diseño se planteó el uso de un campo `fecha_registro` que se actualizara automáticamente mediante triggers cada vez que un registro fuera modificado. Sin embargo, esta idea fue descartada, ya que no se pudo implementar de forma adecuada, pues exigía una mayor complejidad en la parte de la implementación lógica. De cualquier forma se conserva su definición dada la propuesta inicial

```
1 create or replace function s_08_function_actualizar_fecha_administrativo()
2 returns trigger as $$
3 begin
4     update administrativo
5     set fecha_registro = current_date
6     where administrativo_id = new.administrativo_id;
7     return null;
8 end;
9 $$ language plpgsql;
```

35: Trigger para actualizar la fecha del registro automáticamente en administrativo

```
1 create or replace trigger s_08_trigger_actualizacion_fecha_reg_administrativo
2 before update on administrativo for each row
3 execute function s_08_function_actualizar_fecha_administrativo();
```

36: Trigger para actualizar la fecha del registro automáticamente en administrativo

Ejemplo de otro trigger para actualizar la fecha de manipulación del registro:

```
1 create or replace function s_08_function_actualizar_fecha_categoria()
2 returns trigger as $$
3 begin
4     update categoria
5     set fecha_registro = current_date
6     where categoria_id = new.categoria_id;
7     return null;
8 end;
9 $$ language plpgsql;
```

37: Trigger para actualizar la fecha del registro automáticamente en categoria

```
1 create or replace trigger s_08_trigger_actualizacion_fecha_reg_categoria
2 before update on categoria for each row
3 execute function
4     s_08_function_actualizar_fecha_categoria();
```

38: Trigger para actualizar la fecha del registro automáticamente en categoria

Para el resto de tablas se creó un trigger con la misma estructura que los anteriores. Se omite su código para evitar redundancia.

5. Migración de datos

La migración de datos se presentó como una regla de negocio para poder mantener un control de información sobre la base de datos sin necesidad de un intermediario con la finalidad de evitar errores humanos, esto también funciona para auditorías y una posible escalabilidad futura del proyecto. Esto, sin embargo, es un problema que requiere de un análisis para que funcione correctamente. Debido a los alcances del curso y la complejidad para migrar los datos entre tablas, se decidió adoptar la solución de eliminar y recargar las tablas al final de cada día en conjunto con la tecnología de Apache Airflow, sin embargo esta solución es un poco deficiente y se podría sustituir por una inserción de los datos que son registrados o modificados con suficientes conocimientos y tiempo.

Para la segunda base de datos (final) se utilizaron los mismos scripts para la generación de tablas, vistas y programación, de modo que ambas bases fueran gemelas en estructura para facilitar la inserción de datos. Esto se logra exportando las tablas de la base de datos original mediante los siguientes comandos:

Proceso de copiado de tablas de base de datos 1

```

1  \copy public.estado TO 'PATH/restaurante_airflow/data/estado.csv' WITH (FORMAT csv,
    HEADER true, DELIMITER ',')
2  \copy public.categoria TO 'PATH/restaurante_airflow/data/categoria.csv' WITH (FORMAT
    csv, HEADER true, DELIMITER ',')
3  \copy public.empleado TO 'PATH/restaurante_airflow/data/empleado.csv' WITH (FORMAT
    csv, HEADER true, DELIMITER ',')
4  \copy public.cliente TO 'PATH/restaurante_airflow/data/cliente.csv' WITH (FORMAT csv
    , HEADER true, DELIMITER ',')
5  \copy public.producto TO 'PATH/restaurante_airflow/data/producto.csv' WITH (FORMAT
    csv, HEADER true, DELIMITER ',')
6  \copy public.administrativo TO 'PATH/restaurante_airflow/data/administrativo.csv'
    WITH (FORMAT csv, HEADER true, DELIMITER ',')
7  \copy public.cocinero TO 'PATH/restaurante_airflow/data/cocinero.csv' WITH (FORMAT
    csv, HEADER true, DELIMITER ',')
8  \copy public.dependiente TO 'PATH/restaurante_airflow/data/dependiente.csv' WITH (
    FORMAT csv, HEADER true, DELIMITER ',')
9  \copy public.mesero TO 'PATH/restaurante_airflow/data/mesero.csv' WITH (FORMAT csv,
    HEADER true, DELIMITER ',')
10 \copy public.telefono_empleado TO 'PATH/restaurante_airflow/data/telefono_empleado.
    csv' WITH (FORMAT csv, HEADER true, DELIMITER ',')
11 \copy public.orden TO 'PATH/restaurante_airflow/data/orden.csv' WITH (FORMAT csv,
    HEADER true, DELIMITER ',')
12 \copy public.orden_producto TO 'PATH/restaurante_airflow/data/orden_producto.csv'
    WITH (FORMAT csv, HEADER true, DELIMITER ',')
13 \copy public.factura TO 'PATH/restaurante_airflow/data/factura.csv' WITH (FORMAT csv
    , HEADER true, DELIMITER ',')

```

39: Copia de tablas originales a csv

Se utiliza el comando `\copy` para poder exportar las tablas en formato csv para más tarde cargarlos en la segunda base de datos.

Como se comentó anteriormente, la segunda base de datos se elimina y recarga cada día, de igual manera realiza un reajuste de índices para poder mantener una velocidad óptima al momento de realizar consultas mediante los siguientes comandos:

Procedimientos sobre base de datos 2

```
1  \set ON_ERROR_STOP on
2
3  BEGIN;
4
5  TRUNCATE TABLE
6      public.factura,
7      public.orden_producto,
8      public.orden,
9      public.telefono_empleado,
10     public.dependiente,
11     public.administrativo,
12     public.cocinero,
13     public.mesero,
14     public.producto,
15     public.cliente,
16     public.empleado,
17     public.categoria,
18     public.estado
19  RESTART IDENTITY CASCADE;
20
21  \copy public.estado FROM 'PATH/restaurante_airflow/data/estado.csv' WITH (FORMAT csv
22  , HEADER true, DELIMITER ',')
23  \copy public.categoria FROM 'PATH/restaurante_airflow/data/categoria.csv' WITH (
24  FORMAT csv, HEADER true, DELIMITER ',')
25  \copy public.empleado FROM 'PATH/restaurante_airflow/data/empleado.csv' WITH (FORMAT
26  csv, HEADER true, DELIMITER ',')
27  \copy public.cliente FROM 'PATH/restaurante_airflow/data/cliente.csv' WITH (FORMAT
28  csv, HEADER true, DELIMITER ',')
29  \copy public.producto FROM 'PATH/restaurante_airflow/data/producto.csv' WITH (FORMAT
30  csv, HEADER true, DELIMITER ',')
31  \copy public.administrativo FROM 'PATH/restaurante_airflow/data/administrativo.csv'
32  WITH (FORMAT csv, HEADER true, DELIMITER ',')
33  \copy public.cocinero FROM 'PATH/restaurante_airflow/data/cocinero.csv' WITH (FORMAT
34  csv, HEADER true, DELIMITER ',')
35  \copy public.dependiente FROM 'PATH/restaurante_airflow/data/dependiente.csv' WITH (
36  FORMAT csv, HEADER true, DELIMITER ',')
37  \copy public.mesero FROM 'PATH/restaurante_airflow/data/mesero.csv' WITH (FORMAT csv
38  , HEADER true, DELIMITER ',')
39  \copy public.telefono_empleado FROM 'PATH/restaurante_airflow/data/telefono_empleado
40  .csv' WITH (FORMAT csv, HEADER true, DELIMITER ',')
41  \copy public.orden FROM 'PATH/restaurante_airflow/data/orden.csv' WITH (FORMAT csv,
42  HEADER true, DELIMITER ',')
43  \copy public.orden_producto FROM 'PATH/restaurante_airflow/data/orden_producto.csv'
44  WITH (FORMAT csv, HEADER true, DELIMITER ',')
45  \copy public.factura FROM 'PATH/restaurante_airflow/data/factura.csv' WITH (FORMAT
46  csv, HEADER true, DELIMITER ',')
47
48  DO $$
49  DECLARE
50      v_valor bigint;
51  BEGIN
52      SELECT max(estado_id) INTO v_valor FROM estado;
53      IF v_valor IS NOT NULL THEN
54          PERFORM setval('estado_id_seq', v_valor, true);
55      END IF;
56
57      SELECT max(categoria_id) INTO v_valor FROM categoria;
58      IF v_valor IS NOT NULL THEN
```



```

46     PERFORM setval('categoria_id_seq', v_valor, true);
47 END IF;
48
49 SELECT max(empleado_id) INTO v_valor FROM empleado;
50 IF v_valor IS NOT NULL THEN
51     PERFORM setval('empleado_id_seq', v_valor, true);
52 END IF;
53
54 SELECT max(cliente_id) INTO v_valor FROM cliente;
55 IF v_valor IS NOT NULL THEN
56     PERFORM setval('cliente_id_seq', v_valor, true);
57 END IF;
58
59 SELECT max(producto_id) INTO v_valor FROM producto;
60 IF v_valor IS NOT NULL THEN
61     PERFORM setval('producto_id_seq', v_valor, true);
62 END IF;
63
64 SELECT max(telefono_empleado_id) INTO v_valor FROM telefono_empleado;
65 IF v_valor IS NOT NULL THEN
66     PERFORM setval('telefono_empleado_id_seq', v_valor, true);
67 END IF;
68
69 SELECT max(dependiente_id) INTO v_valor FROM dependiente;
70 IF v_valor IS NOT NULL THEN
71     PERFORM setval('dependiente_id_seq', v_valor, true);
72 END IF;
73
74 SELECT max(orden_id) INTO v_valor FROM orden;
75 IF v_valor IS NOT NULL THEN
76     PERFORM setval('orden_id_seq', v_valor, true);
77 END IF;
78
79 SELECT max(orden_producto_id) INTO v_valor FROM orden_producto;
80 IF v_valor IS NOT NULL THEN
81     PERFORM setval('orden_producto_id_seq', v_valor, true);
82 END IF;
83
84 SELECT max(factura_id) INTO v_valor FROM factura;
85 IF v_valor IS NOT NULL THEN
86     PERFORM setval('factura_id_seq', v_valor, true);
87 END IF;
88
89 SELECT max(substring(folio from 'ORD-([0-9]+)')::bigint)
90 INTO v_valor
91 FROM orden
92 WHERE folio ~ '^ORD-[0-9]+$';
93
94 IF v_valor IS NOT NULL THEN
95     PERFORM setval('factura_folio_seq', v_valor, true);
96 END IF;
97 END $$;
98
99 COMMIT;

```

40: Borrado, carga y actualización de índices sobre segunda BD

Los usuarios definidos para la base final son diferentes a los de la original, esto debido a que se planea un uso únicamente para consultas, permitiendo que solo el administrador pueda realizar todas las operaciones sobre la base, mientras que los administrativos únicamente puedan realizar consultas sobre las tablas además de utilizar

las vistas y bloques de programación creados, como se ve a continuación:

```
1
2 create user admin_res_final with
3     password 'admin_res_final'
4     connection limit 2;
5
6
7 grant all privileges on database restaurante_final to admin_res_final;
8 grant all privileges on schema public to admin_res_final;
9 grant all privileges on all tables in schema public to admin_res_final;
10 grant all privileges on all sequences in schema public to admin_res_final;
11 grant all privileges on all functions in schema public to admin_res_final;
12
13
14 create user administrativo_final with
15     password 'administrativo_final'
16     connection limit 3;
17
18 grant connect on database restaurante_final to administrativo_final;
19 grant usage on schema public to administrativo_final;
20 grant select on all tables in schema public to administrativo_final;
21 grant usage on all sequences in schema public to administrativo_final;
22 grant execute on all functions in schema public to administrativo_final;
23 grant execute on all procedures in schema public to administrativo_final;
```

41: Permisos para el administrador sobre la segunda base de datos

Finalmente, para realizar el flujo de trabajo entre las 2 bases de datos se hace uso de un bloque de código de Python y de Apache Airflow, así como los archivos .csv anteriormente generados.

En la primera parte del código se importan las librerías necesarias para construir el flujo de trabajo en Apache Airflow. En particular, se utilizan dag y task para definir el DAG, es decir, el flujo de trabajo, y las tareas individuales que se ejecutarán dentro de dicho flujo.

También se hacen uso de os y subprocess. La librería os permite interactuar con el sistema operativo, por ejemplo, para validar archivos, carpetas o variables del entorno. Por otro lado, subprocess permite ejecutar comandos de terminal desde Python, lo cual se usará para comunicarse con PostgreSQL mediante comandos como psql y ejecutar los scripts SQL encargados de exportar y cargar los datos entre las bases de datos.

```
1 from datetime import datetime
2 from pathlib import Path
3 import os
4 import subprocess
5
6 from airflow.decorators import dag, task
```

42: Declaración de PATHs, archivos y tablas

En esta sección se inicializan las variables de configuración del proyecto, indicando las rutas donde se encuentran almacenados los scripts SQL que se ejecutarán para realizar la exportación y la carga de datos entre ambas bases de datos. También se define la carpeta donde se guardarán los archivos CSV generados durante el proceso, los cuales funcionan como intermediarios para trasladar la información desde la base de datos de origen hacia la base de datos de destino.

Asimismo, se inicializa una lista de cadenas llamada TABLAS, la cual contiene los nombres de las tablas que forman parte de la base de datos. Esta lista se utiliza para iterar sobre cada tabla y verificar que los archivos CSV

correspondientes se hayan generado correctamente durante el proceso de exportación. De esta forma, el DAG puede validar que la información necesaria exista antes de continuar con la carga de datos hacia la base de datos destino.

```
1
2 BASE_DIR = Path("/home/javi/restaurante_airflow")
3 DATA_DIR = BASE_DIR / "data"
4 SQL_DIR = BASE_DIR / "sql"
5
6 EXPORT_SQL = SQL_DIR / "s-07-exportar.sql"
7 LOAD_SQL = SQL_DIR / "s-08-cargar.sql"
8
9 DB_ORIGEN = "restaurante_dia"
10 DB_DESTINO = "restaurante_final"
11
12 TABLAS = [
13     "estado",
14     "categoria",
15     "empleado",
16     "cliente",
17     "producto",
18     "administrativo",
19     "cocinero",
20     "dependiente",
21     "mesero",
22     "telefono_empleado",
23     "orden",
24     "orden_producto",
25     "factura",
26 ]
```

43: Declaración de PATHs, archivos y tablas

La función construir comando psql crea el comando base para conectarse a PostgreSQL desde la terminal usando psql. Recibe el nombre de la base de datos y ejecuta la conexión como usuario postgres.

La función ejecutar comando ejecuta el comando usando subprocess.run. Si el comando funciona correctamente, muestra la salida obtenida; si ocurre un error, lanza una excepción con el detalle del problema.

La función ejecutar archivo sql permite ejecutar un archivo .sql en una base de datos específica. Primero construye el comando de conexión, después agrega el archivo SQL con la opción -f y finalmente lo ejecuta.

```
1
2 def construir_comando_psql(base_datos: str):
3     return [
4         "sudo",
5         "-n",
6         "-u",
7         "postgres",
8         "psql",
9         "-d",
10        base_datos,
11        "-v",
12        "ON_ERROR_STOP=1",
13    ]
14
```



```

15
16 def ejecutar_comando(comando):
17     resultado = subprocess.run(
18         comando,
19         text=True,
20         capture_output=True,
21         env=os.environ.copy(),
22     )
23
24     if resultado.returncode != 0:
25         raise RuntimeError(
26             "Error ejecutando comando:\n"
27             f"COMANDO:\n{comando}\n\n"
28             f"STDOUT:\n{resultado.stdout}\n\n"
29             f"STDERR:\n{resultado.stderr}"
30         )
31
32     print(resultado.stdout)
33     return resultado.stdout
34
35
36 def ejecutar_archivo_sql(base_datos: str, archivo_sql: Path):
37     comando = construir_comando_psql(base_datos)
38     comando.extend(["-f", str(archivo_sql)])
39     return ejecutar_comando(comando)

```

44: Declaración de PATHs, archivos y tablas

En esta parte se crea el DAG llamado dag carga restaurante, el cual representa el flujo de trabajo completo para realizar la carga de datos del restaurante. Se define una fecha de inicio, se indica que se ejecutará automáticamente todos los días con `schedule=@daily` y se desactiva `catchup` para evitar ejecuciones pasadas.

Dentro del DAG se define la función carga restaurante, que contiene las tareas que formarán parte del flujo. Esta función sirve como contenedor principal donde se organizan los pasos que Airflow deberá ejecutar.

La primera tarea se llama preparar directorios. Esta tarea crea los directorios necesarios para almacenar los archivos CSV y los scripts SQL, en caso de que no existan previamente.

Además, la tarea verifica que existan los scripts de exportación y carga. Si alguno de estos archivos no se encuentra, se lanza un error para detener el flujo antes de continuar con el proceso.

Finalmente, se imprimen las rutas de los directorios y scripts utilizados, lo cual permite confirmar en los logs de Airflow que la configuración inicial fue preparada correctamente.

```

1
2 @dag(
3     dag_id="dag_carga_restaurante",
4     start_date=datetime(2026, 1, 1),
5     schedule=@daily,
6     catchup=False,
7     tags=["restaurante", "postgresql", "csv"],
8 )
9 def carga_restaurante():
10
11     @task
12     def preparar_directorios():
13         DATA_DIR.mkdir(parents=True, exist_ok=True)

```



```

14     SQL_DIR.mkdir(parents=True, exist_ok=True)
15
16     if not EXPORT_SQL.exists():
17         raise FileNotFoundError(f"No existe el archivo: {EXPORT_SQL}")
18
19     if not LOAD_SQL.exists():
20         raise FileNotFoundError(f"No existe el archivo: {LOAD_SQL}")
21
22     print(f"Directorio de datos: {DATA_DIR}")
23     print(f"Directorio SQL: {SQL_DIR}")
24     print(f"Script de exportación: {EXPORT_SQL}")
25     print(f"Script de carga: {LOAD_SQL}")
26     print("Directorios preparados correctamente.")

```

45: Declaración de PATHs, archivos y tablas

La tarea exportar csv desde restaurante dia se encarga de exportar los datos desde la base de datos de origen restaurante dia hacia archivos CSV. Para ello, ejecuta el script SQL de exportación mediante la función ejecutar archivo sql.

La tarea verificar csv generados revisa que todos los archivos CSV hayan sido creados correctamente. Para esto, recorre la lista TABLAS y verifica que exista un archivo CSV por cada tabla dentro del directorio de datos.

Si alguno de los archivos CSV no existe, la tarea guarda su ruta en la lista faltantes y lanza un error para detener el flujo. Si todos los archivos fueron generados correctamente, imprime un mensaje de confirmación en los logs de Airflow.

```

1
2 @task
3     def exportar_csv_desde_restaurante_dia():
4         print("Exportando datos desde restaurante_dia hacia archivos CSV...")
5         ejecutar_archivo_sql(DB_ORIGEN, EXPORT_SQL)
6         print("Exportación finalizada correctamente.")
7
8     @task
9     def verificar_csv_generados():
10         faltantes = []
11
12         for tabla in TABLAS:
13             archivo = DATA_DIR / f"{tabla}.csv"
14
15             if not archivo.exists():
16                 faltantes.append(str(archivo))
17
18         if faltantes:
19             raise FileNotFoundError(
20                 "Faltan los siguientes archivos CSV:\n" + "\n".join(faltantes)
21             )
22
23         print("Todos los archivos CSV fueron generados correctamente.")

```

46: Declaración de PATHs, archivos y tablas

La tarea cargar csv en restaurante final se encarga de cargar los archivos CSV generados previamente en la base de datos destino restaurante final. Para ello, ejecuta el script SQL de carga mediante la función ejecutar archivo sql.

Después, se crean las variables inicio, exportar, verificar y cargar, las cuales representan la ejecución de cada tarea definida dentro del DAG. Cada una corresponde a un paso del proceso de migración de datos.



Proyecto Final - Bases de Datos 2026-2

Finalmente, con inicio >> exportar >> verificar >> cargar se establece el orden de ejecución de las tareas. Esto indica que primero se preparan los directorios, después se exportan los CSV, luego se verifican los archivos generados y al final se cargan en la base de datos destino.

La línea dag carga restaurante = carga_restaurante() instancia el DAG, es decir, registra el flujo de trabajo para que Apache Airflow pueda detectarlo y mostrarlo en su interfaz.

```
1 @task
2 def cargar_csv_en_restaurante_final():
3     print("Cargando archivos CSV en restaurante_final...")
4     ejecutar_archivo_sql(DB_DESTINO, LOAD_SQL)
5     print("Carga finalizada correctamente.")
6
7
8     inicio = preparar_directorios()
9     exportar = exportar_csv_desde_restaurante_dia()
10    verificar = verificar_csv_generados()
11    cargar = cargar_csv_en_restaurante_final()
12
13    inicio >> exportar >> verificar >> cargar
14
15
16 dag_carga_restaurante = carga_restaurante()
```

47: Declaración de PATHs, archivos y tablas

Ejemplos de actualización:

A continuación se presenta una serie de capturas mostrando distintas actualizaciones:

```
seville@bd-1ra: /restaurantes/airflow$ sudo -u postgres psql -d restaurante_dia -c "select * from estado"
estado_id | estado | fecha_registro
-----
1 | Ciudad de México | 2026-05-29
2 | Jalisco | 2026-05-29
3 | Nuevo León | 2026-05-30
4 | Puebla | 2026-05-30
(4 rows)

seville@bd-1ra: /restaurantes/airflow$ sudo -u postgres psql -d restaurante_final -c "select * from estado"
estado_id | estado | fecha_registro
-----
1 | Ciudad de México | 2026-05-29
2 | Jalisco | 2026-05-29
3 | Nuevo León | 2026-05-30
4 | Puebla | 2026-05-30
(4 rows)

seville@bd-1ra: /restaurantes/airflow$
```

Figura 3: Estado inicial, ambas bases de datos cuentan con 4 registros en la tabla estado.

```
seville@bd-1ra: /restaurantes/airflow$ sudo -u postgres psql -d restaurante_dia -c "insert into estado values(5,'Oaxaca',default); INSERT 0 1
seville@bd-1ra: /restaurantes/airflow$ sudo -u postgres psql -d restaurante_dia -c "select * from estado"
estado_id | estado | fecha_registro
-----
1 | Ciudad de México | 2026-05-29
2 | Jalisco | 2026-05-29
3 | Nuevo León | 2026-05-30
4 | Puebla | 2026-05-30
5 | Oaxaca | 2026-05-30
(5 rows)

seville@bd-1ra: /restaurantes/airflow$ sudo -u postgres psql -d restaurante_final -c "select * from estado"
estado_id | estado | fecha_registro
-----
1 | Ciudad de México | 2026-05-29
2 | Jalisco | 2026-05-29
3 | Nuevo León | 2026-05-30
4 | Puebla | 2026-05-30
(4 rows)

seville@bd-1ra: /restaurantes/airflow$
```

Figura 4: Inserción de un nuevo estado en restaurante_dia.

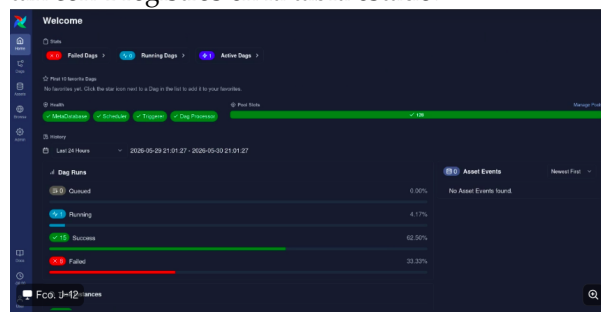


Figura 5: Disparo manual del DAG desde la interfaz de Apache Airflow.

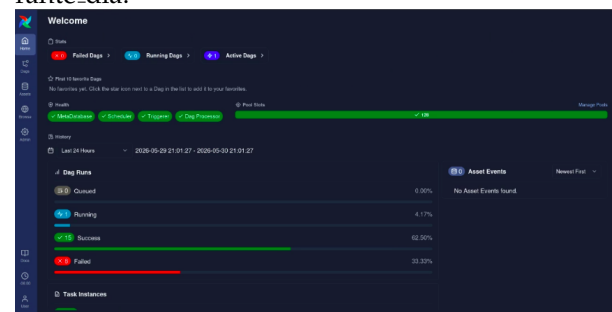


Figura 6: Vista general del DAG con historial de ejecuciones y registro de tareas fallidas anteriores.

Proyecto Final - Bases de Datos 2026-2

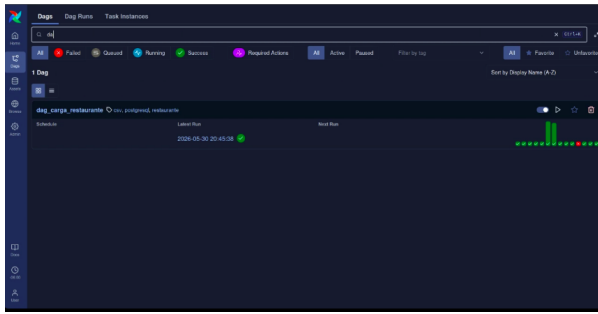


Figura 7: Lista de DAGs registrados en Airflow.

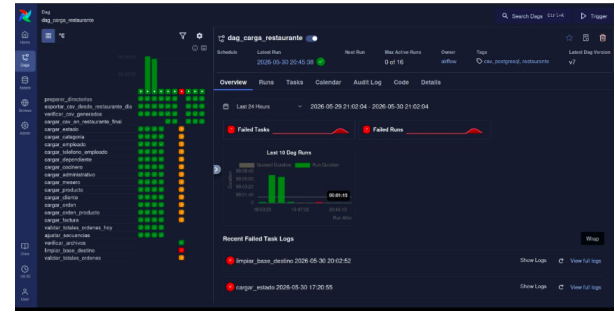


Figura 8: Pantalla de inicio de Airflow mostrando el resumen de ejecuciones de las últimas 24 horas.

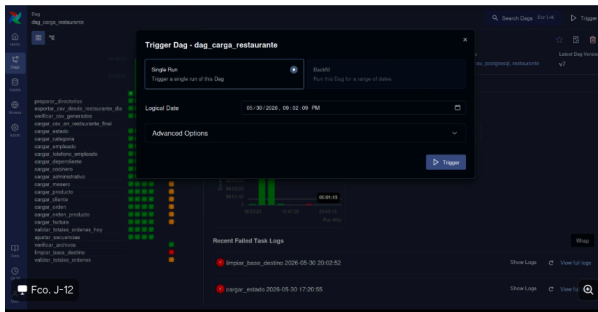


Figura 9: Confirmación de salud del sistema.

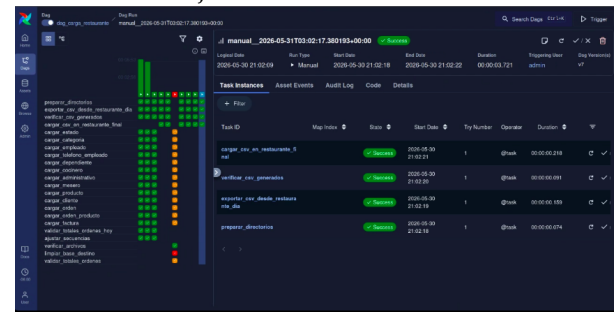


Figura 10: Ejecución exitosa del DAG, las cuatro tareas del flujo finalizaron con estado *Success*.

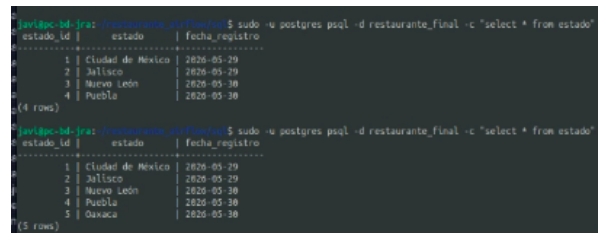


Figura 11: Estado final, restaurante_final refleja el nuevo registro, confirmando la migración exitosa.



6. Conclusiones

Arias Jiménez Alejandro:

El proyecto plantea requerimientos que engloban todos los conocimientos necesarios sobre Bases de Datos. Estos requerimientos en su mayoría se cumplieron de manera correcta, sin embargo, pienso que la orquestación podría haber sido un poco más laboriosa con tal de mantener todos los registros para poder realizar auditorías y llevar un mejor flujo de trabajo.

Con respecto a los objetivos específicos, considero que casi todos los puntos se cubrieron de manera completa gracias a que los requerimientos del proyecto fueron bastante similares a los ejercicios en clase y los vistos en los exámenes, por lo que se pudo realizar con facilidad. Esto también ayudó a reforzar todos los conocimientos que adquirí en clases, principalmente el modelado de una base de datos a partir de un texto, así como la manipulación de los datos mediante procedimientos, triggers y funciones para lograr una automatización y reducir el error humano. También me ayudó a practicar consultas y subconsultas, las cuales pueden ser un poco complejas si no se tiene la práctica ni el conocimiento suficiente.

El único problema que encontré fue el de orquestar la comunicación entre 2 bases de datos, que a mi parecer es un tema muy importante, pues sé que se utilizan en la industria como una base de datos principal y otra que se utiliza para consultas, lo cual puede resultar benéfico para poder realizar estudios con todos los datos, predecir fenómenos y esto, como se dijo al inicio del curso, puede ayudar a tomar una decisión más informada. Como lo dije, creo que fue un problema por falta de conocimientos y la premura de la entrega, junto con otras variables, no nos permitió llevar este punto a cabo en su completitud, y en cambio tuvimos que optar por una solución más rudimentaria, que si bien cumple con su objetivo, puede gastar mucha memoria y procesos, lo que no es tan recomendable, principalmente sabiendo que eso pone mucho estrés en ambas bases de datos.

Calderon Almanza Murat:

La realización del proyecto planteado fue exitosa en su mayoría, pues se cumplieron todos los requerimientos solicitados y considero que de una manera eficiente en la mayoría.

Desde mi punto de vista el caso de estudio planteado fue muy similar a los vistos en clase, por lo que no representó un desafío muy grande el entendimiento y solución del mismo, al igual que las especificaciones solicitadas para el proyecto.

El mayor desafío presentado fue la migración de datos entre dos bases, pues a diferencia del resto de requerimientos, no contábamos con experiencia o información alguna que nos ayudara como equipo a resolver dicha problemática, esto desencadenó en una solución poco eficiente, que tal vez con mayor contexto en este ámbito o conocimiento sobre estas herramientas y su manipulación harían del proyecto final una versión mucho más sólida y eficiente.

Personalmente disfruté de la realización del proyecto y esperaba un mayor reto en los requerimientos solicitados para SQL, sin embargo, pienso que la experiencia brindada por el mismo es bastante rica, dando una introducción a los proyectos de bases de datos, que si bien en la industria son a una escala muy diferente sigue englobando la mayoría de conceptos básicos vistos a lo largo del curso.

Cruz Trejo Dan Benjamín:

Este proyecto fue de gran ayuda para materializar lo aprendido en el curso de Bases de Datos, pues requirió utilizar el conocimiento adquirido de todos los temas e implementarlo en un problema de la vida real. Considero que las principales dificultades fueron lograr implementar herramientas y estrategias para poder llevar a cabo el trabajo de forma colaborativa. Al contar con diferentes posibles soluciones al problema, se necesitó una adecuada comunicación para llegar a un acuerdo y establecer la decisión final. De igual forma, la parte dos del proyecto representó una gran dificultad, pues no sentí que contáramos con los conocimientos necesarios para resolverlo. Aún así, con una buena investigación fue más que suficiente para poder abordarlo. Considero que esto terminó siendo una gran experiencia, ya que es bastante parecido a lo que sucede en el mundo real, lo que nos permite tener una buena preparación académica para cuando entremos al mundo laboral.

De la O Vicente Christopher Giovanni:

Este proyecto final representó un gran reto para poner en práctica los conceptos vistos a lo largo del curso, desde el análisis y modelado hasta la implementación física de la base de datos. La primera parte, relacionada con el diseño del modelo entidad-relación, su transformación al modelo relacional y la implementación de los scripts DDL, no presentó mayores complicaciones, ya que se siguió una metodología estructurada que permitió cumplir con las reglas de negocio, la normalización y las restricciones de integridad. Las dificultades surgieron principalmente en la segunda parte, donde fue necesario investigar de manera autónoma cómo orquestar la carga de datos desde archivos de texto hacia una segunda base de datos, manejando la extracción de la información, implementando mecanismos para evitar duplicados y gestionando excepciones para validar el flujo del proceso. A pesar de los desafíos, se logró un resultado funcional que cumple con lo solicitado. Esta experiencia evidenció el verdadero alcance de una base de datos más allá del almacenamiento simple, mostrando temas como la integración con lenguajes externos, el uso de archivos como medio de intercambio de información y la automatización de procesos de migración, aspectos que sin duda pueden ser explorados con mayor profundidad en el futuro.

Campos Cortés Isaac Jareth:

El desarrollo de este proyecto resultó ser una manera muy interesante de aplicar de manera práctica los conceptos aprendidos a lo largo del curso. La necesidad de comprender los temas para asegurar la cohesión del proyecto. Además, todo el proceso fue construyendo en etapas anteriores, por lo que fue necesario realizar un análisis integral desde un inicio.

Desde mi punto de vista logramos de manera exitosa plantear e implementar una solución para el caso de estudio propuesto. La gran organización y trabajo en equipo fueron cruciales para asegurar que cada sección implementada fuera adecuada.

A pesar de esto, encontramos una serie de desafíos durante el proceso, el más evidente fue al momento de implementar el requerimiento de la segunda parte para transferir los datos de la base original a la secundaria. Por una falta de conocimiento sobre el uso de Apache Airflow fue complicado realizar la implementación como habíamos pensado originalmente. Sin embargo, fuimos capaces de realizar una implementación, aunque no haya sido la más elegante. Este es el aspecto con mayor capacidad de mejora de todo el proyecto.

En general considero que el proyecto fue completado de la mejor manera posible con nuestros conocimientos y presentó aprendizajes invaluable sobre la implementación de manera tangible de todo lo aprendido durante el curso y en el campo de nuestras habilidades blandas al trabajar en equipo.



Francisco Javier Reyes Abarca:

En conclusión, la primera parte del proyecto permitió aplicar los conceptos principales del diseño de bases de datos, comenzando con el análisis de requerimientos y continuando con la elaboración del modelo Entidad-Relación y el modelo relacional.

La implementación en PostgreSQL permitió transformar el diseño en una base de datos funcional mediante el uso de tablas, llaves primarias y foráneas, restricciones, secuencias, vistas, índices, funciones y triggers. Estos elementos ayudaron a mantener la integridad de los datos y a automatizar reglas de negocio importantes, como validar la disponibilidad de productos y calcular correctamente los totales de las órdenes.

Finalmente, la parte de Airflow complementó el proyecto al automatizar la migración de datos entre la base de datos operativa y la base final. Mediante el uso de archivos CSV, tareas de exportación, verificación y carga, se construyó un flujo controlado y repetible. Aunque la solución recarga toda la información, representa una base sólida para futuras mejoras, como implementar cargas incrementales más eficientes.